PROJECT REPORT

Enterprise SIEM Lab: Deployed a Wazuh SIEM Cluster to Aggregate Logs and Create Custom Alert Dashboards

Project domain: Security Information and Event Management (SIEM)
Platform: Wazuh
Institution branding: Sandip University logo supplied for this report

Abstract

This project describes an enterprise-style SIEM lab using Wazuh to collect and correlate security events from authorized endpoints, investigate alerts, and present operational dashboards. A companion Python utility processes exported Wazuh alert data, normalizes common fields, summarizes severity/rule/agent counts, and generates JSON and CSV outputs for dashboard preparation and triage. The included script is not a Wazuh deployment installer and does not connect to a live cluster; actual collection and dashboard configuration are performed within the separately deployed lab.

1. Introduction

Security teams need a central view of events from servers, workstations, and applications. A SIEM platform aggregates logs, applies rules, and helps analysts identify suspicious activity. Wazuh provides security monitoring capabilities including log analysis, alerting, endpoint monitoring, and a web dashboard. A lab environment helps students understand collection pipelines, alert tuning, triage, and dashboard design without affecting production systems.

2. Problem Statement

When system logs remain distributed across endpoints, analysts may miss related activity or spend excessive time reviewing events manually. A centralized SIEM and consistent alert summary process can improve visibility, reduce triage effort, and make it easier to identify recurring rules, noisy agents, and higher-priority events.

3. Objectives

- Design a lab architecture with Wazuh manager, indexer, dashboard, and authorized endpoint agents.
- Aggregate security-relevant logs into a central analysis platform.
- Review alert severity, rule triggers, and source agents.
- Create dashboard-oriented summaries and a CSV triage queue from exported alerts.
- Document safe validation, alert tuning, and incident-review practices.

4. Scope and Safety

The lab should run in an isolated virtual network and monitor only approved endpoints. Generate controlled test events using dedicated lab accounts and benign actions. Protect dashboard credentials and log exports; logs may contain personal data, hostnames, IP addresses, or secrets. The included Python utility reads local exports only and does not change Wazuh configuration or execute actions on endpoints.

5. Proposed Architecture

Component	Function
Endpoint agents	Collect configured event logs and security telemetry from authorized Windows/Linux lab hosts.

Component	Function
Wazuh manager	Receives events, applies decoders and rules, and produces alerts.
Wazuh indexer	Stores and indexes alert/event data for searching and analysis.
Wazuh dashboard	Provides visual exploration, alert views, and operational dashboards.
Python reporting utility	Processes exported alerts into JSON summaries and a CSV triage queue.

6. Methodology

1) Create an isolated virtual lab and define its network and access boundaries. 2) Deploy Wazuh components using official version-specific documentation. 3) Enroll approved endpoint agents and confirm connectivity. 4) Configure collection for relevant operating-system and application logs. 5) Generate benign test events in the lab and verify that expected alerts appear. 6) Review rule levels, agent names, timestamps, and descriptions in the dashboard. 7) Export alert data as JSON/JSONL and run the Python reporting utility. 8) Tune noisy rules carefully, record false positives, and repeat the test.

7. Python Utility Design

The utility accepts a Wazuh alert export in JSON or JSON Lines format. It normalizes common nested fields such as rule.id, rule.level, rule.description, agent.id, and agent.name, along with timestamps and log messages. It calculates counts by severity, level, rule, and agent, then writes a dashboard-friendly JSON summary and CSV triage queue. A minimum rule-level filter can reduce the output to events selected for closer review.

8. Example Execution

```
python enterprise_siem_lab.py --input sample_wazuh_alerts.json
python enterprise_siem_lab.py --input alerts.json --min-level 7 --json-output summary.json --csv-output t
```

The sample records are synthetic and intended only to demonstrate the workflow. Replace them with sanitized exports from an authorized lab when collecting actual project evidence.

9. Dashboard and Alert Use Cases

Useful dashboard panels include alerts over time, counts by rule level, top triggering rules, alerts by agent, and a triage queue for high-priority records. Example lab scenarios include repeated failed logons, file integrity changes, and unusual web-server requests. These are illustrative scenarios; exact alert behavior depends on installed rules, log sources, and configuration.

10. Expected Outcomes

- Centralized visibility of monitored lab events through Wazuh.
- Repeatable alert review and triage workflow.
- Structured JSON summaries and CSV records for reporting.
- Improved understanding of rule tuning, false positives, log retention, and incident-response documentation.

11. Limitations

The Python script is a companion report processor, not a full SIEM. It does not deploy or configure Wazuh, connect to a manager API, build dashboards inside Wazuh, or guarantee that all alert fields are present. The sample data is synthetic. Alert severity mapping is a convenience approximation and must be adapted to organizational triage policy. Production use requires access controls, retention planning, monitoring, backups, and operational validation.

12. Conclusion

The Enterprise SIEM Lab provides a practical framework for learning centralized log monitoring with Wazuh. By combining a properly isolated Wazuh deployment with a lightweight Python alert summarizer, the project demonstrates how security events can be collected, reviewed, prioritized, and documented. Real deployment evidence should include sanitized screenshots, configuration notes, and actual lab results.

References and Further Reading

- Official Wazuh documentation — installation, agents, log data collection, rules, and dashboards.
- NIST SP 800-92 — Guide to Computer Security Log Management.
- MITRE ATT&CK; — technique references for contextualizing observed behavior.
- Organization-specific incident response and log-retention policies.

Prepared as an educational project template. Add screenshots and verified results from your own authorized Wazuh lab before presenting it as a completed deployment.